



ANYWORK365 / FINANCIAL SYSTEMS

Wallet, Internal Locked-Funds Release & Payment Architecture

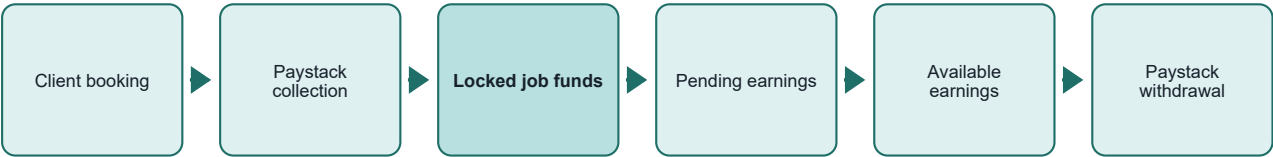
A production architecture for booking-specific Paystack collections, immutable internal accounting, artisan earnings release, refunds, withdrawals, disputes and financial operations.

TERMINOLOGY	The product uses locked job funds, pending earnings and available earnings. This is the internal mechanism the team has informally called escrow; the document does not make a regulated custody claim.
-------------	---

RAIL	SOURCE OF TRUTH	CURRENCY
Paystack for external payment collection, refunds and bank transfers.	Anywork365's immutable, balanced internal ledger.	NGN only, recorded as integer kobo - never floating point.

Executive architecture

The wallet is a marketplace accounting view, not an unrestricted stored-value or user-to-user transfer product.



Booking-specific collection The server creates the booking and exact payment amount before Paystack initialization. There is no general top-up.	Availability states Locked, pending, held and withdrawal-pending money cannot be spent or withdrawn.	One ledger boundary Only LedgerService may post financial entries or update account projections.
Provider verification Amount, currency, environment, customer email and booking metadata must all match.	Safe external calls A durable intent or reservation commits before any Paystack network request.	Operationally reconcilable Provider events, audit logs, outbox records and internal references provide end-to-end traceability.

NON-NEGOTIABLE BOUNDARY

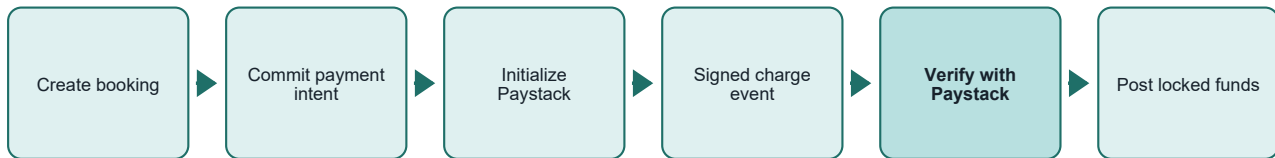
Clients cannot send arbitrary cash-like value to artisans. Money reaches an artisan only through a funded booking and an authorized release event.

System roles

Role	Financial responsibility	Explicit restriction
Client	Creates and pays for a booking; confirms completion.	Cannot transfer arbitrary value to another user.
Artisan	Accepts a funded booking; receives held earnings.	Cannot withdraw pending, held or disputed earnings.
Paystack	External collection, refund and bank-transfer rail.	Does not define internal spendability or booking state.
Finance admin	Approves exceptions and reconciliation with scoped permission.	Cannot directly edit account balances or posted entries.
Support	Views user progress and transaction context.	No financial mutation or finance permission.

Payment collection architecture

A payment intent is authoritative because it is created from the server-side booking, not from an amount supplied to a wallet top-up endpoint.



Successful collection sequence

1. The server validates the client, artisan, booking description, date and exact NGN price.
2. One transaction commits the booking, job-funds record, versioned fee rule and unique payment intent.
3. Paystack is initialized only after that commit, using the same reference, amount and booking metadata.
4. The callback improves user experience; signed webhooks provide the durable asynchronous confirmation path.
5. Verification must return SUCCESS and match reference, amount, NGN, environment, customer email, booking ID and client UID.
6. One idempotent journal moves external payment clearing into the booking's locked-funds account.

Collection journal	Signed amount	Meaning
External payment clearing	- NGN 100,000.00	Provider collection recognized by the internal system.
Booking locked job funds	+ NGN 100,000.00	Client funds become non-spendable and tied to this booking.
Journal total	NGN 0.00	Required invariant for every posted transaction.

LATE PAYMENT AFTER CANCELLATION

If cancellation is requested while Paystack is still processing, the job enters CANCEL_REQUESTED. A later successful payment is still verified and posted, then immediately reserved for refund. It is never orphaned.

Duplicate webhook Unique event hash plus posting idempotency returns the existing result.	Changed replay Reusing an idempotency key with different parameters returns a conflict.	Unknown provider result No internal credit is posted until Paystack verification succeeds.
---	---	--

Internal locked-funds release

Completion never credits an immediately withdrawable balance. It first creates pending artisan earnings and recognizes the versioned platform commission.



Release authorization and locking

Authorization Only the paying client may mark a confirmed booking complete.	Concurrency The booking and job-funds rows are locked before checking release state.
Exactly once The release idempotency key is derived from the immutable job-funds record.	Fee traceability The fee rule version and calculated amount were fixed when funding was created.

Release journal example	Signed amount	Availability after posting
Booking locked job funds	- NGN 100,000.00	Booking account returns to zero.
Artisan pending earnings	+ NGN 95,000.00	Not withdrawable during the safety hold.
Platform commission revenue	+ NGN 5,000.00	Versioned 500-basis-point example.
Journal total	NGN 0.00	Balanced before commit.

HOLD RELEASE

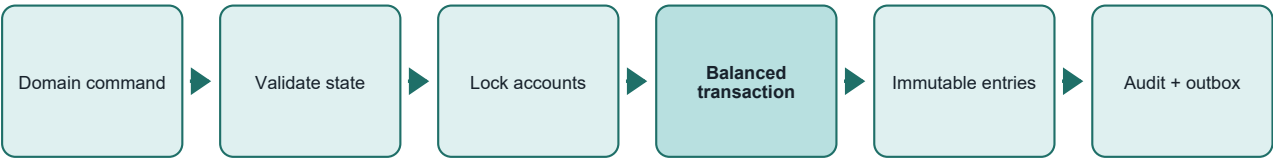
A protected worker selects matured holds with row locks and SKIP LOCKED. It posts pending earnings to available earnings exactly once, then emits a notification through the transactional outbox.

State progression

State	Can be spent?	Can be withdrawn?	Exit condition
LOCKED JOB FUNDS	No	No	Authorized completion or refund decision.
PENDING EARNINGS	No	No	Safety hold matures without dispute.
RISK HOLD	No	No	Finance or dispute outcome.
AVAILABLE EARNINGS	No P2P	Yes	Withdrawal reservation.
WITHDRAWAL PENDING	No	No	Verified provider terminal status.

Ledger and account architecture

The journal is the source of truth. Cached account balances exist only as row-locked projections and are continuously reconciled.



Core invariants

At least two accounts A posted business event cannot contain a one-sided entry.	Zero-sum by currency All consolidated NGN entry deltas must sum to exactly zero.	Integer minor units Application calculations use bigint and database amounts use BIGINT kobo.
Immutable posting Database triggers block UPDATE and DELETE on posted transactions and entries.	Nonnegative protection User, booking and revenue accounts cannot cross below zero unless explicitly allowed.	Compensating corrections Errors are reversed by a new linked event; history is never rewritten.

Account families

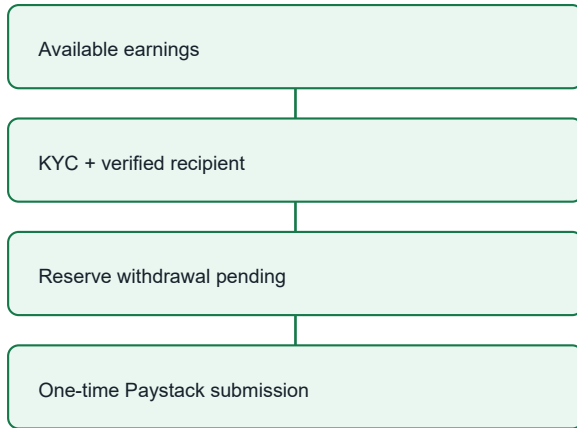
Owner	Representative accounts	Classification / policy
Client	Available marketplace funds; locked job funds; refund pending; refundable funds	LIABILITY; nonnegative
Artisan	Pending; available; withdrawal pending; withdrawn; reversed; reserve hold	LIABILITY; nonnegative
Platform	Commission; transaction fee; refund/chargeback liability; operational reserve	REVENUE, LIABILITY or ASSET
System	External payment/transfer clearing; suspense; opening balance; adjustment	CLEARING, SUSPENSE or EQUITY; explicitly controlled

ACCOUNT CREATION	Public APIs cannot create arbitrary financial accounts. Account construction is allow-listed in the internal account registry and always fixes owner, purpose, classification, currency and negative-balance policy.
-------------------------	--

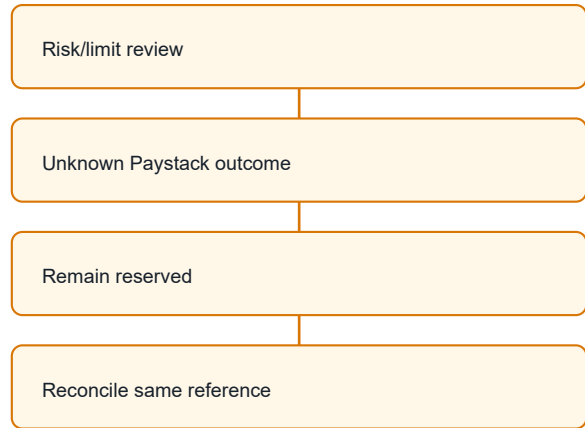
Artisan withdrawals and money movement

Funding and withdrawals use Paystack; internal booking release remains an internal ledger movement. Available earnings are the only withdrawable balance.

SUCCESS PATH



EXCEPTION PATH



Control	Implemented behavior	Failure behavior
Identity	Verified profile plus NIN interim gate.	Request denied.
Bank ownership	Resolved Paystack name must match verified profile tokens.	Recipient rejected or manual review.
Bank change	Configurable 24-hour default delay.	Withdrawal denied until age threshold.
Velocity	Minimum, maximum, daily and monthly NGN limits.	Request denied before reservation.
Risk	Active holds block; threshold can require finance approval.	Funds remain available until a valid reservation.
Submission	Persistent 16-50 character internal reference and one database claim.	Unknown result remains reserved; never auto-resubmitted.
Finalization	Verify amount, currency, environment and provider state.	Success -> withdrawn; failed/reversed -> returned.

DOUBLE-SPEND DEFENSE

Concurrent requests lock the available account and withdrawal workflow record. The first valid reservation reduces available earnings; a competing request sees the reduced balance and cannot overdraw it.

Withdrawal journals

Event	Decrease	Increase
Reserve request	Artisan available earnings	Artisan withdrawal pending
Provider success	Artisan withdrawal pending	Artisan withdrawn earnings
Provider failed/reversed	Artisan withdrawal pending	Artisan available earnings

Webhook, idempotency and reliability

The public webhook does not synchronously perform financial side effects in v3. It verifies, persists and acknowledges; a worker performs idempotent domain processing.



Reliability mechanism	What it prevents	Storage/control
Payload hash uniqueness	The same signed event being processed twice.	provider_events unique provider + hash
Provider ID uniqueness	Alternate duplicate delivery where an ID is supplied.	provider_events unique provider + event ID
Request hash	Same idempotency key reused with a changed amount or target.	financial_idempotency_records
Posting key	Duplicate money movement across callback and webhook.	money_transactions unique idempotency key
Processing lease	Two workers processing the same event concurrently.	FOR UPDATE SKIP LOCKED + token
Bounded retry	Silent transient failure or infinite retry loops.	attempt count, backoff and dead letter
Transactional outbox	Notification side effects being lost before commit.	financial_outbox_events

Provider events explicitly handled

Collections charge.success	Transfers transfer.success / failed / reversed
Refunds pending / processing / needs-attention / failed / processed	Disputes charge.dispute.create

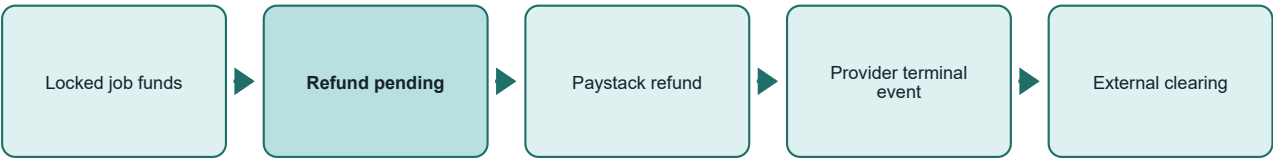
FAIL-CLOSED RULE

Invalid signatures are rejected. Signed but unknown event types are retained and marked ignored. Repeated failures move to dead letter and require an operational alert.

Refunds, disputes and chargebacks

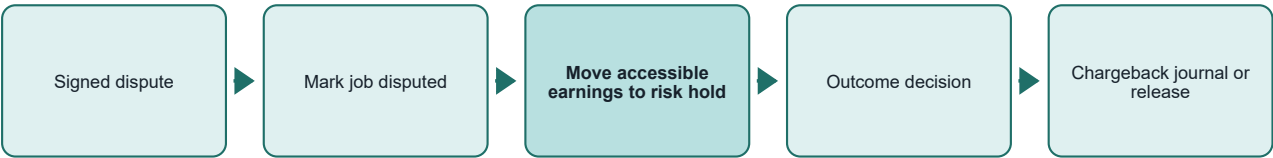
External reversals are modeled as explicit workflows and balanced journals. No posted history is edited.

Refund flow



Refund state	Internal money state	Required action
REQUESTED / PROCESSING	Client refund-pending remains non-spendable.	Wait for signed Paystack state.
NEEDS_ATTENTION	Reservation remains intact.	Finance verifies customer details and follows Paystack retry procedure.
PROCESSED	Refund pending decreases; external payment clearing increases.	Notify client and reconcile.
FAILED	Refund pending moves to client refundable funds.	Finance decides safe retry or future-booking use.

Dispute and chargeback flow



LOSS ALLOCATION

A lost dispute consumes locked job funds if unreleased. After release, it consumes the artisan risk hold and related commission first; any remaining shortfall is recorded against the platform operational reserve. Compliance and finance must approve this policy.

Data model, controls and reporting

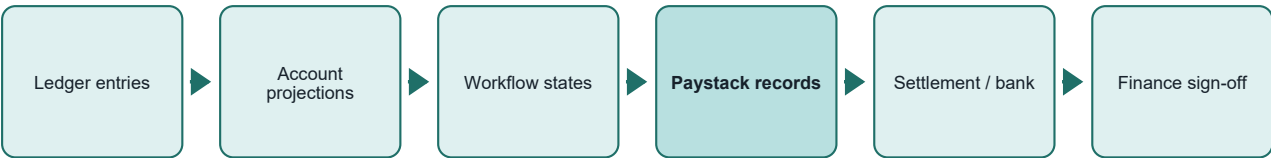
Workflow tables explain business state; the immutable journal explains every monetary movement.

Domain area	Primary records	Key protections
Ledger	money_accounts, money_transactions, money_entries	Balanced journal, nonnegative policy, immutable posting
Collection	marketplace_payment_intents, provider_events	Unique references, exact verification, durable ingestion
Booking funds	job_funds, platform_fee_rules, earnings_holds	One row per booking, versioned fee, timed release
Withdrawals	transfer_recipients, marketplace_withdrawal_requests	Masked account, ownership, reservation, single submission
Refund/risk	refund_requests, financial_disputes, risk_holds, financial_chargebacks	Explicit state and compensating entries
Operations	financial_audit_logs, financial_outbox_events, reconciliation_items	Attribution, reliable side effects, exception evidence
Access	financial_admin_permissions, financial_adjustments	Least privilege, reason, ticket and balanced adjustment

Reports derived from immutable records

User statements Complete transaction history with reference and related booking.	Artisan earnings Gross release, fee, pending-to-available events and withdrawals.
Client payments Intent, provider reference, booking, amount and status.	Platform finance Commission, refunds, failures, daily summary and reconciliation variance.

Reconciliation layers



NO DIRECT BALANCE EDITS

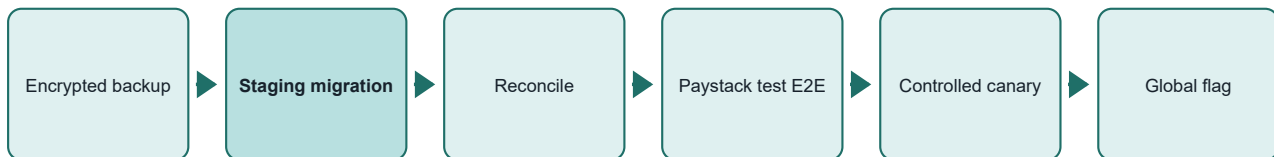
Finance corrections require a typed reversal or a permission-gated adjustment with an idempotency key, reason and ticket. Support remains view-only.

Production posture and activation gates

The architecture is implemented, type-checked and production-built, but the v3 schema and feature flag remain intentionally inactive.

Gate	Current status	Required completion
Application architecture	IMPLEMENTED	Review and commit the working tree.
Type and build validation	PASSED	Repeat in deployment CI.
Financial unit/property tests	9 PASSED / 1 SKIPPED	Run MySQL concurrency test against an isolated test database.
V3 migration	DRY RUN PASSED	Apply first to staging; repeat and reconcile.
Paystack E2E	NOT RUN	Exercise payment, duplicate event, transfer, refund and dispute in test mode.
Worker operations	NOT CONFIGURED	Provision secret, scheduler, uptime and dead-letter alerts.
Finance permissions	SCHEMA READY	Grant only to named MFA-protected finance users.
Compliance	OPEN	Approve characterization, KYC/AML, terms, tax and dispute policy.

Controlled activation sequence



CURRENT DECISION

Keep `MARKETPLACE_FINANCE_V3_ENABLED=false`. Do not apply the v3 production migration or enable live flows until staging, Paystack configuration, finance operations and compliance gates are signed off.

Required Paystack operations

- Configure the production HTTPS webhook URL and verify signed test delivery.
- Confirm registered-business transfer eligibility, approval mode and sufficient transfer/refund balance.
- Separate test and live keys in the deployment secret manager; restrict dashboard access and require MFA.
- Reconcile internal references against Paystack payments, refunds, transfers and settlement records.

Architecture decisions and references

The design prioritizes financial correctness, traceability and safe failure over preserving unsafe legacy wallet behavior.

IMPLEMENTED Booking-specific ledger, release states, withdrawals, refunds, risk, events, outbox and reporting.	TESTED Type-check, property/invariant tests, migration dry run and optimized Next.js build.
MIGRATION READY Additive and checksummed; staging rehearsal is still mandatory.	BUSINESS DECISION Fee, hold, limits, refundable-fund reuse and controlled-canary policy.
PAYSTACK CONFIGURATION Webhook, transfer approval, balance, settlement, roles and MFA.	COMPLIANCE CONFIRMATION Custody characterization, KYC/AML, tax, customer terms and incident duties.

Repository source documents

Document	Repository path
Architecture	docs/wallet-architecture.md
Business rules	docs/wallet-business-rules.md
State machines	docs/wallet-state-machines.md
Paystack boundary	docs/paystack-integration.md
Migration runbook	docs/wallet-migration-runbook.md
Reconciliation runbook	docs/wallet-reconciliation-runbook.md
Incident response	docs/financial-incident-response.md
Final implementation report	docs/wallet-overhaul-final-report.md

External technical references

- Paystack Webhooks - <https://paystack.com/docs/payments/webhooks/>
- Paystack Verify Payments - <https://paystack.com/docs/payments/verify-payments/>
- Paystack Single Transfers - <https://paystack.com/docs/transfers/single-transfers/>
- Paystack Refunds - <https://paystack.com/docs/payments/refunds/>

DOCUMENT CONTROL	This PDF describes the implementation present in the local Anywork365 workspace on 28 July 2026. It is an architecture and operating-control document, not legal advice or evidence of production activation.
-------------------------	---